

WHAT IT IS

A counterparty intelligence platform where compliance officers upload vendor lists, a deterministic engine resolves each name against Sayari's commercial world model, traverses beneficial ownership, and screens OFAC SDN — then a grounded Claude co-pilot narrates the result with every claim cited to a source. ~13,000 lines across Python, React, and SQL. Production architecture from day one; no LLM in the data path.

WHY IT MATTERS TO SAYARI

- Built directly in Sayari's data domain — corporate/trade relationships, beneficial ownership, sanctions screening.
- Demonstrates the FDE pattern: a customer-grounded application that feeds the product flywheel.
- Platform thinking, not just app thinking — the same six tools serve a human UI and any MCP client.
- Honest scope: production-grade provenance + defended architectural decisions; hardening scoped for the next pass.

ARCHITECTURE IN ONE PARAGRAPH

A React frontend (Babel-standalone, no build step) is served by a Python FastAPI service that exposes the engine as typed tools, streams Server-Sent Events for the upload pipeline and the co-pilot, and holds the in-memory OFAC matcher and credential store. An importable pure-Python engine does resolution, profile extraction, and caching; a Python MCP server imports that same engine directly to expose six tools to Claude Desktop. External calls reach the Sayari SDK, a locally-indexed OFAC SDN feed (17,207 rows), and Anthropic. Persistence is file-first — every value is a real cached Sayari response, committed and greppable — with an optional six-table Postgres path the code falls back from. Designed for customer-deployed containers in the customer's own cloud tenant: defensible for FINMA-grade compliance customers.